

Module 5 – Events and AWT

Event handling is fundamental to Java programming because it is integral to the creation of many kinds of applications. Events are supported by a number of packages, including `java.util`, `java.awt`, and `java.awt.event`. Most events to which the program will respond are generated when the user interacts with a GUI-based program.

The Delegation Event Model:

The approach to handling events is based on the delegation event model, which defines standard and consistent mechanisms to generate and process events. Its concept is quite simple: a source generates an event and sends it to one or more listeners. In this scheme, the listener simply waits until it receives an event. Once an event is received, the listener processes the event and then returns. The advantage of this design is that the application logic that processes events is cleanly separated from the user interface logic that generates those events. A user interface element is able to “delegate” the processing of an event to a separate piece of code. In the delegation event model, listeners must register with a source in order to receive an event notification. This provides an important benefit: notifications are sent only to listeners that want to receive them.

Events:

In the delegation model, an event is an object that describes a state change in a source. An event can be generated as a consequence of a person interacting with the elements in a graphical user interface. Some of the activities that cause events to be generated are pressing a button, entering a character via the keyboard, selecting an item in a list, and clicking the mouse. Events may also occur that are not directly caused by interactions with a user interface. For example, an event may be generated when a timer expires, a counter exceeds a value, a software or hardware failure occurs, or an operation is completed.

Event Source:

A source is an object that generates an event. This occurs when the internal state of that object changes in some way. Sources may generate more than one type of event. A source must register listeners in order for the listeners to receive notifications about a specific type of event. Each type of event has its own registration method.

The general form is: `public void addTypeListener (TypeListener el )`

Here, `Type` is the name of the event, and `el` is a reference to the event listener. For example, the method that registers a keyboard event listener is called `addKeyListener( )`. The method that registers a mouse motion listener is called `addMouseMotionListener( )`. When an event occurs, all registered listeners are notified and receive a copy of the event object. This is known as multicasting the event. In all cases, notifications are sent only to listeners that register to receive them.

Some sources may allow only one listener to register. In such cases an event occurs, the only one registered listener is notified. This is known as unicasting the event.

A source also provides a method that allows a listener to unregister an interest in a specific type of event.

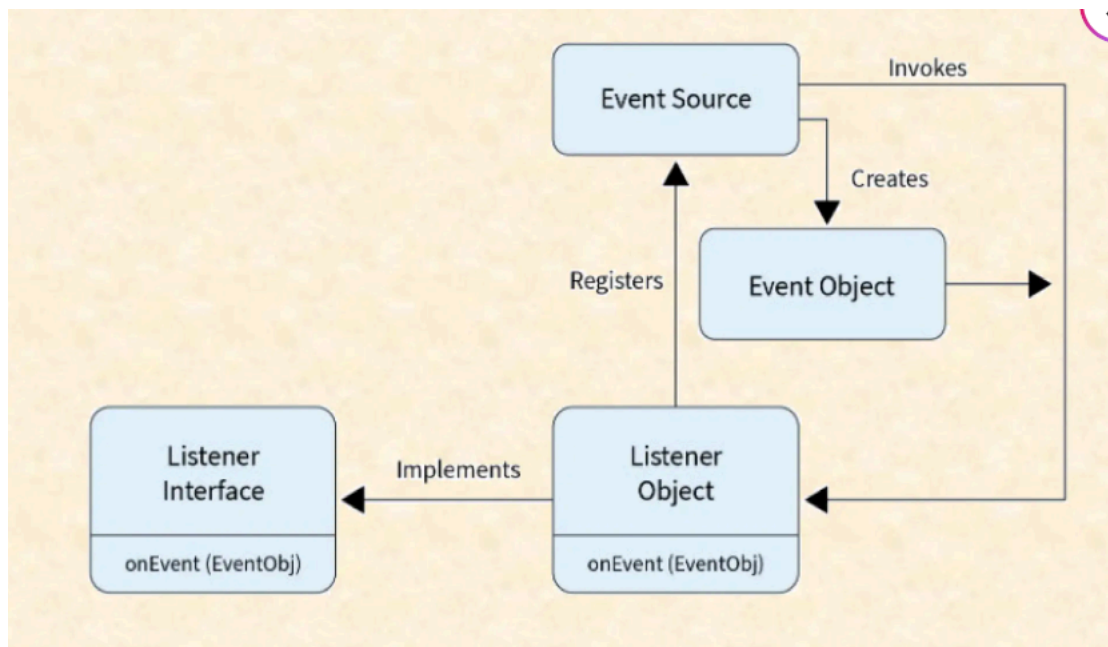
The general form of such a method is: `public void removeTypeListener(TypeListener el )`

Here, Type is the name of the event, and el is a reference to the event listener. For example, to remove a keyboard listener, one would call `removeKeyListener( )`.

Event Listeners:

A listener is an object that is notified when an event occurs. It has two major requirements. First, it must have been registered with one or more sources to receive notifications about specific types of events. Second, it must implement methods to receive and process these notifications. The methods that receive and process events are defined in a set of interfaces, such as those found in `java.awt.event`. For example, the `MouseMotionListener` interface defines two methods to receive notifications when the mouse is dragged or moved. Any object may receive and process one or both of these events if it provides an implementation of this interface.

A schematic representation of the Delegation Event Model is given below:



Event classes:

At the root of the Java event class hierarchy is `EventObject`, which is in `java.util`. It is the superclass for all events.

Constructor: `EventObject(Object src)` Here, src is the object that generates this event.

`EventObject` defines two methods: `getSource( )` and `toString( )`. The `getSource( )` method returns the source of the event and `toString( )` returns the string equivalent of the event. The class `AWTEvent`, defined within the `java.awt` package, is a subclass of `EventObject`. It is the superclass of all AWT-based events.

The MouseEvent Class:

There are eight types of mouse events. The MouseEvent class defines the following integer constants that can be used to identify them:

MOUSE_CLICKED	The user clicked the mouse.
MOUSE_DRAGGED	The user dragged the mouse.
MOUSE_ENTERED	The mouse entered a component.
MOUSE_EXITED	The mouse exited from a component.
MOUSE_MOVED	The mouse moved.
MOUSE_PRESSED	The mouse was pressed.
MOUSE_RELEASED	The mouse was released.
MOUSE_WHEEL	The mouse wheel was moved.

Commonly used methods of the MouseEvent class:

int getX() int getY() - return the X and Y coordinates of the mouse within the component when the event occurred.

Point getPoint() - method to obtain the coordinates of the mouse.

void translatePoint(int x, int y) - changes the location of the event. The arguments x and y are added to the coordinates of the event.

int getClickCount() - obtains the number of mouse clicks for this event.

The MouseListener Interface:

This interface defines five methods. If the mouse is pressed and released at the same point, mouseClicked() is invoked. When the mouse enters a component, the mouseEntered() method is called. When it leaves, mouseExited() is called. The mousePressed() and mouseReleased() methods are invoked when the mouse is pressed and released, respectively.

The general forms of these methods are given below:

void mouseClicked(MouseEvent me)

void mouseEntered(MouseEvent me)

void mouseExited(MouseEvent me)

void mousePressed(MouseEvent me)

void mouseReleased(MouseEvent me)

The MouseMotionListener Interface: This interface defines two methods. The mouseDragged() method is called multiple times as the mouse is dragged. The mouseMoved() method is called multiple times as the mouse is moved. Their general forms are given below:

void mouseDragged(MouseEvent me)

void mouseMoved(MouseEvent me)

Using the Delegation Event Model:

1. Implement the appropriate interface in the listener so that it can receive the type of event desired.
2. Implement code to register and unregister (if necessary) the listener as a recipient for the event notifications.

Note: A source may generate several types of events. Each event must be registered separately. Also, an object may register to receive several types of events, but it must implement all of the interfaces that are required to receive these events.

Program showing implementation of MouseListener and MouseMotionListener:

```
import java.awt.*;
import java.awt.event.*;

class MouseEvents extends Frame
{
    implements MouseListener, MouseMotionListener {

        String msg = "";
        int mouseX = 0, mouseY = 0;

        MouseEvents() {
            setTitle("Mouse Events Demo");
            setSize(300, 200);
            setVisible(true);

            addMouseListener(this);
            addMouseMotionListener(this);

            // Close window properly
            addWindowListener(new WindowAdapter() {
                public void windowClosing(WindowEvent e) {
                    System.exit(0);
                }
            });
        }
    }
}
```

```

// Handle mouse pressed
public void mousePressed(MouseEvent me) {
    mouseX = me.getX();
    mouseY = me.getY();
    msg = "Down";
    repaint();
}

// Handle mouse released
public void mouseReleased(MouseEvent me) {
    mouseX = me.getX();
    mouseY = me.getY();
    msg = "Up";
    repaint();
}

// Handle mouse dragged
public void mouseDragged(MouseEvent me) {
    mouseX = me.getX();
    mouseY = me.getY();
    msg = "*";
    repaint();
}

// Handle mouse moved
public void mouseMoved(MouseEvent me) {
    setTitle("Moving at " + me.getX() + ", " + me.getY());
}

public void paint(Graphics g) {
    g.drawString(msg, mouseX, mouseY);
}

public static void main(String[] args) {
    new MouseEvents();
}
}

```

```

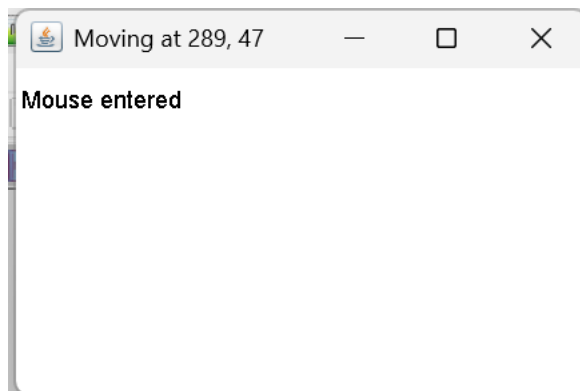
//Handle mouse clicked
public void mouseClicked(MouseEvent me) {
    mouseX = 10;
    mouseY = 50;
    msg = "Mouse clicked";
    repaint();
}

// Handle mouse entered
public void mouseEntered(MouseEvent me) {
    mouseX = 10;
    mouseY = 50;
    msg = "Mouse entered";
    repaint();
}

// Handle mouse exited
public void mouseExited(MouseEvent me) {
    mouseX = 10;
    mouseY = 50;
    msg = "Mouse exited";
    repaint();
}

```

Output:



Abstract Window Toolkit:

The AWT defines a window and adds functionality and specificity. The two most common windows are those derived from Panel, which is used by applets, and those derived from Frame, which creates a standard application window.

Component:

At the top of the AWT hierarchy is the Component class. Component is an abstract class that encapsulates all of the attributes of a visual component. Except for menus, all user interface elements that are displayed on the screen and that interact with the user are subclasses of Component. It defines over a hundred public methods that are responsible for managing events, such as mouse and keyboard input, positioning and sizing the window, and repainting. A Component object is responsible for remembering the current foreground and background colors and the currently selected text font.

Container:

The Container class is a subclass of Component. It has additional methods that allow other Component objects to be nested within it. Other Container objects can be stored inside of a Container. A container is responsible for laying out (that is, positioning) any components that it contains. It does this through the use of various layout managers.

Panel: The Panel class is a concrete subclass of Container. A Panel may be thought of as a recursively nestable, concrete screen component. Panel is the superclass for Applet. When screen output is directed to an applet, it is drawn on the surface of a Panel object. In essence, a Panel is a window that does not contain a title bar, menu bar, or border.

Frame: Frame encapsulates what is commonly thought of as a “window.” It is a subclass of Window and has a title bar, menu bar, borders, and resizing corners. The precise look of a Frame will differ among environments.

The AWT includes several methods that support graphics. All graphics are drawn relative to a window. The origin of each window is at the top-left corner and is 0,0. Coordinates are specified in pixels. The Graphics class defines a number of methods that draw various types of objects, such as lines, rectangles, and arcs. In several cases, objects can be drawn edge-only or filled. Objects are drawn and filled in the currently selected color, which is black by default. When a graphics object is drawn that exceeds the dimensions of the window, output is automatically clipped.

Methods to draw graphical objects

To draw a line: `void drawLine(int startX, int startY, int endX, int endY)`

`drawLine( )` displays a line in the current drawing color that begins at `startX`, `startY` and ends at `endX`, `endY`.

To draw rectangle: `void drawRect(int left, int top, int width, int height)`

`void fillRect(int left, int top, int width, int height)`

The upper-left corner of the rectangle is at `left`, `top`. The dimensions of the rectangle are specified by `width` and `height`.

To draw rounded rectangle:

`void drawRoundRect(int left, int top, int width, int height, int xDiam, int yDiam)`

`void fillRoundRect(int left, int top, int width, int height, int xDiam, int yDiam)`

A rounded rectangle has rounded corners. The upper-left corner of the rectangle is at left, top. The dimensions of the rectangle are specified by width and height. The diameter of the rounding arc along the X axis is specified by xDiam. The diameter of the rounding arc along the Y axis is specified by yDiam.

To draw an ellipse:

```
void drawOval(int left, int top, int width, int height)
```

To fill an ellipse:

```
void fillOval(int left, int top, int width, int height)
```

The ellipse is drawn within a bounding rectangle whose upper-left corner is specified by left, top and whose width and height are specified by width and height. To draw a circle, specify a square as the bounding rectangle.

To draw an arc and filled arc:

```
void drawArc(int left, int top, int width, int height, int startAngle, int sweepAngle)
```

```
void fillArc(int left, int top, int width, int height, int startAngle, int sweepAngle)
```

The arc is bounded by the rectangle whose upper-left corner is specified by left, top and whose width and height are specified by width and height. The arc is drawn from startAngle through the angular distance specified by sweepAngle. Angles are specified in degrees. Zero degrees is on the horizontal, at the three o'clock position. The arc is drawn counterclockwise if sweepAngle is positive, and clockwise if sweepAngle is negative. Therefore, to draw an arc from twelve o'clock to six o'clock, the start angle would be 90 and the sweep angle 180.

To draw figures:

```
void drawPolygon(int x[ ], int y[ ], int numPoints)
```

```
void fillPolygon(int x[ ], int y[ ], int numPoints)
```

The polygon's endpoints are specified by the coordinate pairs contained within the x and y arrays. The number of points defined by these arrays is specified by numPoints. There are alternative forms of these methods in which the polygon is specified by a Polygon object. Below is a program that uses all the above functions and draws lines and figures.


```
import java.awt.*;
import java.awt.event.*;

class GraphicsDemoAWT extends Frame {

    GraphicsDemoAWT() {
        setTitle("AWT Graphics Demo");
        setSize(400, 700);
        setVisible(true);

        // Close window properly
        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                System.exit(0);
            }
        });
    }

    // Override paint() to draw graphics
    public void paint(Graphics g) {
        // Draw lines
        g.drawLine(0, 0, 100, 90);
        g.drawLine(0, 90, 100, 10);
        g.drawLine(40, 25, 250, 80);
    }
}
```

```

// Draw rectangles
g.drawRect(10, 150, 60, 50);
g.fillRect(100, 150, 60, 50);
g.drawRoundRect(190, 150, 60, 50, 15, 15);
g.fillRoundRect(280, 150, 60, 50, 30, 40);

// Draw ellipses / circles
g.drawOval(10, 250, 50, 50);
g.fillOval(90, 250, 75, 50);
g.drawOval(190, 260, 100, 40);

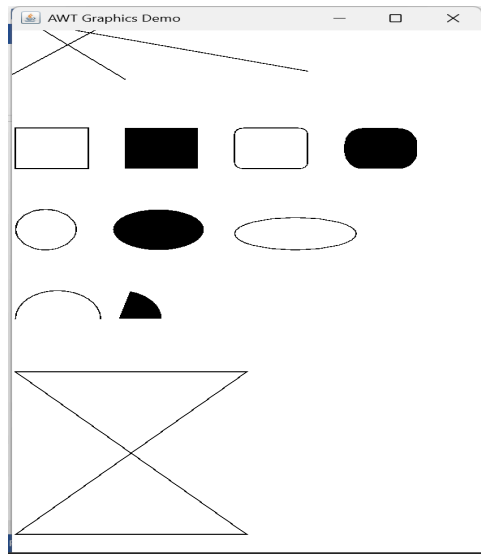
// Draw arcs
g.drawArc(10, 350, 70, 70, 0, 180);
g.fillArc(60, 350, 70, 70, 0, 75);

// Draw polygon
int xpoints[] = {10, 200, 10, 200, 10};
int ypoints[] = {450, 450, 650, 650, 450};
int num = 5;
g.drawPolygon(xpoints, ypoints, num);
}

public static void main(String[] args) {
    new GraphicsDemoAWT();
}
}

```

Output:



The AWT color system allows one to specify any color needed. Example program that work on colors is given below.

```
class ColourDemo extends Frame {  
  
    ColourDemo() {  
        setTitle("AWT Colours Demo");  
        setSize(400, 300);  
        setVisible(true);  
  
        addWindowListener(new WindowAdapter() {  
            public void windowClosing(WindowEvent e) {  
                System.exit(0);  
            }  
        });  
    }  
}
```

```

public void paint(Graphics g) {

    g.setColor(Color.RED);
    g.drawString("Red Color Text", 50, 60);

    g.setColor(Color.BLUE);
    g.drawLine(50, 80, 300, 80);

    g.setColor(Color.GREEN);
    g.drawRect(50, 100, 100, 60);

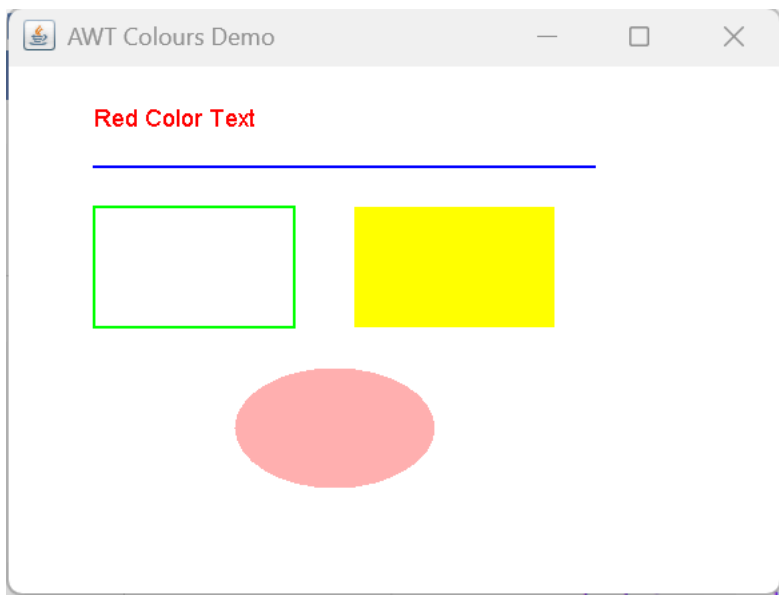
    g.setColor(Color.YELLOW);
    g.fillRect(180, 100, 100, 60);

    g.setColor(Color.PINK);
    g.fillOval(120, 180, 100, 60);
}

public static void main(String[] args) {
    new ColourDemo();
}
}

```

Output:



Program to draw a train using AWT graphics:

```
import java.awt.*;
import java.awt.event.*;

class SimpleTrain extends Frame {

    SimpleTrain() {
        setTitle("Simple Train Drawing");
        setSize(600, 300);
        setVisible(true);

        // Close window properly
        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                System.exit(0);
            }
        });
    }
}
```

```
public void paint(Graphics g) {  
  
    // Engine body  
    g.setColor(Color.BLUE);  
    g.fillRect(50, 120, 150, 60);  
  
    // Chimney  
    g.setColor(Color.DARK_GRAY);  
    g.fillRect(60, 70, 20, 50);  
  
    //Cloud  
    g.setColor(Color.LIGHT_GRAY);  
    g.fillOval(80, 60, 30, 20);  
    g.fillOval(100, 55, 35, 25);  
    g.fillOval(125, 60, 30, 20);  
  
    // Wheels of engine  
    g.setColor(Color.BLACK);  
    g.fillOval(60, 170, 30, 30);  
    g.fillOval(150, 170, 30, 30);  
}
```

```

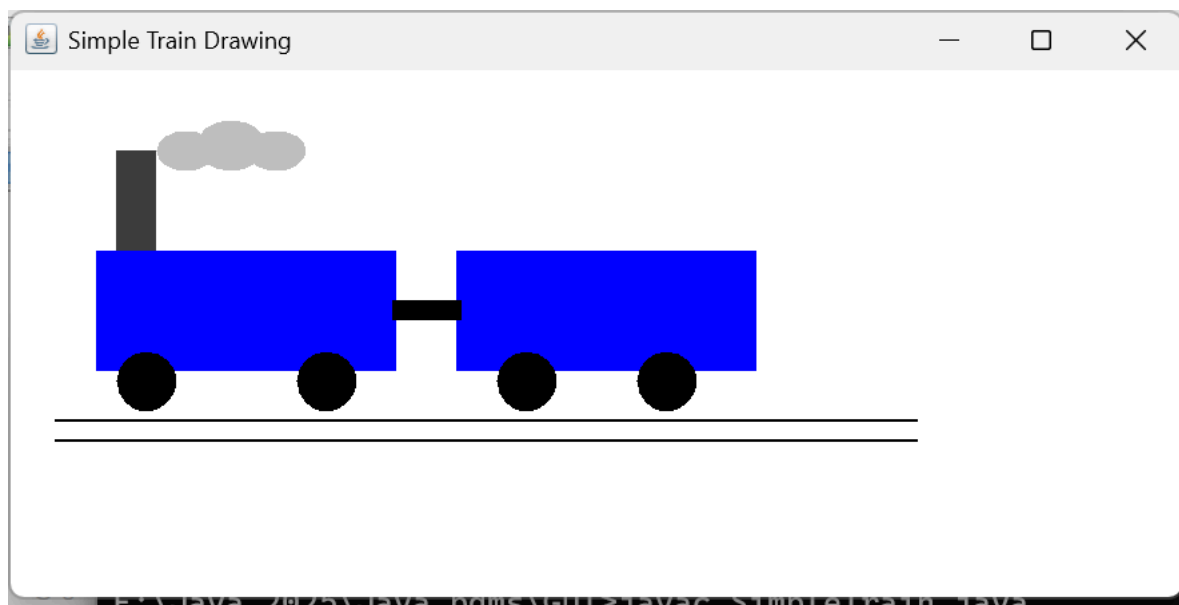
        // Coach body
        g.setColor(Color.BLUE);
        g.fillRect(230, 120, 150, 60);

        // Coach wheels
        g.setColor(Color.BLACK);
        g.fillOval(250, 170, 30, 30);
        g.fillOval(320, 170, 30, 30);

        //Link
        g.setColor(Color.BLACK);
        g.fillRect(198, 145, 35, 10);
        // Track
        g.drawLine(30, 205, 460, 205);
        g.drawLine(30, 215, 460, 215);
    }

    public static void main(String[] args) {
        new SimpleTrain();
    }
}

```



Program to draw Smiley using AWT Graphics:

```
import java.awt.*;
import java.awt.event.*;

class SmileyAWT extends Frame {

    SmileyAWT() {
        setTitle("Smiley");
        setSize(300, 300);
        setVisible(true);

        // Close window properly
        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                System.exit(0);
            }
        });
    }

    public void paint(Graphics g) {

        // Face
        g.setColor(Color.YELLOW);
        g.fillOval(50, 50, 200, 200);

        // Eyes
        g.setColor(Color.BLACK);
        g.fillOval(95, 100, 20, 20);    // left eye
        g.fillOval(165, 100, 20, 20);   // right eye

        // Triangle nose (polygon)
        int x[] = {140, 130, 150};
        int y[] = {130, 160, 160};
        g.fillPolygon(x, y, 3);

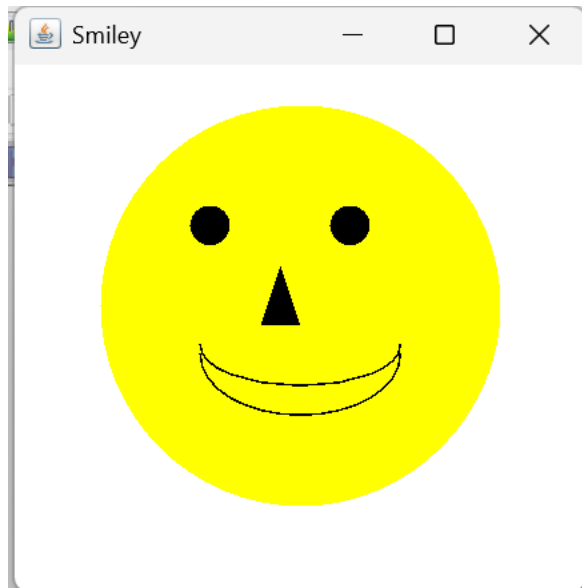
        // Smile
        g.setColor(Color.BLACK);
        g.drawArc(100, 145, 100, 60, 180, 180); // outer arc
        g.drawArc(100, 150, 100, 40, 180, 180); // inner arc
    }
}
```



```

    public static void main(String[] args) {
        new SmileyAWT();
    }
}

```



Program to draw a hut with AWT Graphics:

```

import java.awt.*;
import java.awt.event.*;

class HutHouse extends Frame {

    HutHouse() {
        setTitle("Hut Shaped House");
        setSize(500, 400);
        setVisible(true);

        // Close window properly
        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                System.exit(0);
            }
        });
    }
}

```

```

public void paint(Graphics g) {

    // House base (rectangle)
    g.setColor(Color.ORANGE);
    g.fillRect(150, 180, 200, 140);

    // Roof (triangle / hut shape)
    g.setColor(Color.DARK_GRAY);
    int xRoof[] = {140, 250, 360};
    int yRoof[] = {180, 90, 180};
    g.fillPolygon(xRoof, yRoof, 3);

    // Door
    g.setColor(Color.BLACK);
    g.fillRect(235, 240, 30, 80);

    // Window
    g.setColor(Color.WHITE);
    g.fillRect(175, 220, 40, 40);
    g.setColor(Color.BLACK);
    g.drawRect(175, 220, 40, 40);

    // Ground line
    g.drawLine(50, 320, 450, 320);

    // Sun
    g.setColor(Color.YELLOW);
    g.fillOval(380, 40, 60, 60);
}

public static void main(String[] args) {
    new HutHouse();
}
}

```

